R-Drop Training and Checkpoint Averaging for a CPU-Bounded Causal Decoder

Yu Qinghao
Student ID: 3036483599
DASE7506 Mini Project 1

Protocol `7506-mp1-wt2-v2` Final test BPB: 1.493095

Abstract

This work develops a compact language model under three hard inference limits: at most 64 MiB of assets, 4 GiB peak RAM, and five times the baseline CPU scoring time. The final predictor combines a 10,596,160-parameter causal Transformer, a call-local exact-match cache, temperature calibration, 19,200-step R-Drop training, and uniform averaging of four late checkpoints. It reaches **1.475055 validation BPB** and **1.493095 test BPB**, improving the prior Iteration 13 test result by 0.116060 BPB. A paired validation audit measures 29.492 s versus 6.799 s for the baseline (**4.34**×). The 42.946 MiB checkpoint and a sampled 1.5508 GiB working set pass their gates. Checkpoint averaging contributes 0.005348 validation BPB over the raw final update. The proposed 1.30–1.40 range was not reached.

1 Task and Controls

The benchmark uses a train-fitted 2,048-token BPE vocabulary and independent causal windows of 256 tokens. Every target after the first token is scored once. For summed negative log likelihood L and complete raw UTF-8 byte count B,

$$\mathrm{BPB} = \frac{L}{B \ln 2}. \tag{1}$$

Validation contains 376,599 targets and 1,148,007 bytes; test contains 428,405 targets and 1,292,013 bytes. Architecture and hyperparameter choices used validation. Final test evaluation occurred after the Iteration 14 method and averaged weights were fixed. Raw course data are excluded from the repository.

2 Development Path

2.1 Phase I: Micro-Scale Architecture and Memory

Iterations 1–11 deliberately used models near 1.1M parameters and a matched 1,200-step budget to compare ideas cheaply. RoPE, RMSNorm, and SwiGLU jointly reduced validation BPB from the 2.0710 baseline to 1.828568. An inference-only control applied the exact-match cache to unchanged weights and reduced BPB from 1.828568 to 1.753179. This 0.075389 gain established the cache independently of training.

2.2 Phase II: Scaling and the Architectural Diet

Iteration 12 scaled to eight 320-wide blocks while retaining RoPE and SwiGLU. Its saved checkpoint did not satisfy the paired CPU gate. Iteration 13 replaced SwiGLU with a two-linear-layer 4× GELU MLP and RoPE with learned absolute positions. The resulting inference backbone has eight unshared blocks, width 320, ten 32-dimensional heads, MLP width 1,280, RMSNorm, tied embeddings, and 10,596,160 parameters. This grouped *architectural diet* reduced parameter count and CPU work; the experiment does not isolate the individual timing contribution of either replacement.

2.3 Phase III: Cache and Calibration

At position t, the cache finds strictly earlier positions $j < t$ with the longest available matching suffix: trigram, then bigram, then unigram. A match votes for observed successor x_{j+1} with

$$w(t,j) = \exp\left(-\frac{t-j}{512}\right), \qquad q_t(v) = \frac{\sum_{j \in M_t} w(t,j)\mathbf{1}\{x_{j+1} = v\}}{\sum_{j \in M_t} w(t,j)}. \tag{2}$$

The cache share α_t is 0.60, 0.35, or 0.10 for trigram, bigram, or unigram matches. Since $j + 1 \leq t$, every voted token is already observed. State is recreated for every call. The cache has no trained parameters or neural matrix multiplications, although its comparisons, exponentials, and accumulation require CPU work.

Backbone logits use validation-selected temperature $T = 1.10$:

$$p_t(v) = (1 - \alpha_t)\,\mathrm{softmax}(z_t/T)_v + \alpha_t q_t(v). \tag{3}$$

On the Iteration 13 checkpoint, the neural model at $T = 1.0$ scored 1.643535 validation BPB; calibration alone scored 1.635771, cache alone scored 1.601842, and both scored 1.594317. This was an inference-only 2×2 ablation with identical weights.

3 Iteration 14: R-Drop and Weight Averaging

The final run started from random initialization and processed 157,286,400 training targets in 19,200 updates at batch size 32. AdamW used weight decay 0.1, 100 warmup updates, and cosine decay from 10^{-3} to 10^{-4} through update 19,200. Training took 3,150.09 s on an NVIDIA RTX 4060 Laptop GPU with BF16.

R-Drop requires stochastic views, so dropout 0.1 was added to attention and MLP residual branches during training only. For forward distributions p_1, p_2 and targets y, the loss is

$$\mathcal{L} = \tfrac{1}{2}[\mathrm{CE}(y, p_1) + \mathrm{CE}(y, p_2)] \tag{4}$$

$$+ \tfrac{1}{2}[\mathrm{KL}(p_1 \| p_2) + \mathrm{KL}(p_2 \| p_1)]. \tag{5}$$

The KL coefficient is 1.0. Evaluation bypasses dropout, preserving a deterministic inference graph and parameter count. Checkpoints were saved at updates 17,760, 18,240, 18,720, and 19,200. The averaging script uniformly averages every floating-point parameter in FP32. The raw final update scored 1.480403 validation BPB; averaging improved this to **1.475055**, a gain of 0.005348.

4 Results and Resource Audit

Table 1: Final quality and resource measurements. Lower BPB is better.

Measure	Iteration 13	Iteration 14	Gate / reference
Validation BPB	1.594317	**1.475055**	minimize
Test BPB	1.609155	**1.493095**	minimize
Parameters	10,596,160	**10,596,160**	—
CPU validation	—	**29.492 s**	6.799 s baseline; $4.34\times \leq 5\times$
CPU test	19.471 s	**34.212 s**	complete test run
Checkpoint	40.448 MiB	**42.946 MiB**	≤ 64 MiB
Sampled RAM	—	**1.5508 GiB**	≤ 4 GiB

The final test score improves on Iteration 13 by 0.116060 BPB. The validation timing comparison uses the same host, PyTorch build, thread count, and split. CPU timings varied with system load, making the paired ratio more useful than comparison with an earlier absolute time. RAM was sampled from the evaluator process tree at roughly 200 ms intervals and may miss shorter spikes.

The final checkpoint SHA-256 is `6703409e2705750f546d18c10465c0fbc26c268fa749c9d43900dc49192e155f`. The predictor source SHA-256 is `5d2b65057a2a9a22e9009d575444cb3659074a163e1386e92d722a6c7e50d213`.

5 Interpretation and Limitations

Long training plus R-Drop produces the largest improvement in the project, while late checkpoint averaging adds a smaller measured gain. Training duration, dropout, and R-Drop changed together relative to Iteration 13, so their

individual effects are not identified. Only raw-update versus averaged-checkpoint performance is isolated within Iteration 14. The cache performs quadratic comparisons within each 256-token window. Repeated development on validation introduces selection bias. No external training text, teacher model, pretrained weights, test-answer cache, or cross-window state was used.

The peer PDF named in the task was unavailable in the workspace. Its reported 1.456 BPB and ablation values could not be independently checked and are not treated as measured results. The achieved score is above 1.456 and above the proposed 1.30–1.40 target.

6 Reproduction

With the original course `code/data/` directory installed, run from `code/`:

```
python -m unittest discover -s tests -v
python evaluate.py --checkpoint runs/iter_14/checkpoint.pt \
  --device cpu --precision fp32 --threads 32 --split validation
python evaluate.py --checkpoint runs/iter_14/checkpoint.pt \
  --device cpu --precision fp32 --threads 32 --split test
```

Training uses the committed configuration:

```
python train.py --implementation student --config configs/iter_14.json \
  --device cuda --precision bf16 --threads 4 --seed 17 --steps 19200 \
  --cosine-horizon 19200 --batch-size 32 --rdrop-alpha 1.0 \
  --save-steps 17760,18240,18720,19200 --run-dir runs/iter_14_reproduction
python average_checkpoints.py --run-dir runs/iter_14_reproduction
```

References

[1] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel Mixture Models," 2016.

[2] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," NeurIPS, 2019.

[3] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units," 2016.

[4] X. Liang et al., "R-Drop: Regularized Dropout for Neural Networks," NeurIPS, 2021.

[5] P. Izmailov et al., "Averaging Weights Leads to Wider Optima and Better Generalization," UAI, 2018.

WikiText-2 contains text from Wikipedia contributors. Raw data are excluded. Substantive AI assistance contributed implementation, experiment execution, audits, analysis, and drafting; the author reviewed the final materials.